

eka15_week7_Neural_network

March 13, 2026

Task 1) Logistic Regression Baseline

```
[2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, log_loss

RANDOM_STATE = 42
```

```
[3]: df = pd.read_csv("heart_failure_clinical_records_dataset.csv")

print("Dataset shape:", df.shape)
df.head(10)
```

Dataset shape: (299, 13)

```
[3]:
```

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	\
0	75.0	0	582	0	20	
1	55.0	0	7861	0	38	
2	65.0	0	146	0	20	
3	50.0	1	111	0	20	
4	65.0	1	160	1	20	
5	90.0	1	47	0	40	
6	75.0	1	246	0	15	
7	60.0	1	315	1	60	
8	65.0	0	157	0	65	
9	80.0	1	123	0	35	

	high_blood_pressure	platelets	serum_creatinine	serum_sodium	sex	\
0	1	265000.00	1.9	130	1	
1	0	263358.03	1.1	136	1	

2	0	162000.00	1.3	129	1
3	0	210000.00	1.9	137	1
4	0	327000.00	2.7	116	0
5	1	204000.00	2.1	132	1
6	0	127000.00	1.2	137	1
7	0	454000.00	1.1	131	1
8	0	263358.03	1.5	138	0
9	1	388000.00	9.4	133	1

	smoking	time	DEATH_EVENT
0	0	4	1
1	0	6	1
2	1	7	1
3	0	7	1
4	0	8	1
5	1	8	1
6	0	10	1
7	1	10	1
8	0	10	1
9	1	10	1

```
[4]: X = df.drop(columns=["DEATH_EVENT"])
y = df["DEATH_EVENT"]

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=RANDOM_STATE
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    stratify=y_temp,
    random_state=RANDOM_STATE
)
print("Train:", X_train.shape)
print("Validation:", X_val.shape)
print("Test:", X_test.shape)
```

```
Train: (209, 12)
Validation: (45, 12)
Test: (45, 12)
```

```
[5]: numeric_features = X.columns.tolist()

preprocessor = ColumnTransformer(
    transformers=[("num", StandardScaler(), numeric_features)]
)

log_reg_model = Pipeline([("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))])

log_reg_model.fit(X_train, y_train)

train_pred = log_reg_model.predict(X_train)
val_pred = log_reg_model.predict(X_val)
test_pred = log_reg_model.predict(X_test)

train_prob = log_reg_model.predict_proba(X_train)[: ,1]
val_prob = log_reg_model.predict_proba(X_val)[: ,1]
test_prob = log_reg_model.predict_proba(X_test)[: ,1]

results = {
    "Train Accuracy": accuracy_score(y_train, train_pred),
    "Validation Accuracy": accuracy_score(y_val, val_pred),
    "Test Accuracy": accuracy_score(y_test, test_pred),

    "Train ROC-AUC": roc_auc_score(y_train, train_prob),
    "Validation ROC-AUC": roc_auc_score(y_val, val_prob),
    "Test ROC-AUC": roc_auc_score(y_test, test_prob),

    "Train LogLoss": log_loss(y_train, train_prob),
    "Validation LogLoss": log_loss(y_val, val_prob),
    "Test LogLoss": log_loss(y_test, test_prob)
}

for k,v in results.items():
    print(f"{k}: {v:.4f}")
```

```
Train Accuracy: 0.8517
Validation Accuracy: 0.8444
Test Accuracy: 0.8222
Train ROC-AUC: 0.9062
Validation ROC-AUC: 0.9178
Test ROC-AUC: 0.8065
Train LogLoss: 0.3568
Validation LogLoss: 0.3607
Test LogLoss: 0.5015
```

Task1-Analysis

Logistic regression is a linear model because it computes a weighted linear combination of the input

features before applying the sigmoid function:

$$z = \mathbf{w}^T \mathbf{x} + b$$

Then linear score is converted into a probability using the sigmoid function. As the decision boundary depends on this linear combination of features, logistic regression can only model linear relationships between inputs and the target.

Training and Validation Behaviour

The logistic regression model shows stable behaviour because the training and validation metrics are similar, indicating good generalisation. In contrast, the MLP shows a larger gap between training and validation performance during training, suggesting the possibility of overfitting.

description:

The neural network can capture more complex nonlinear patterns because of its hidden layers and activation functions. However, this increased flexibility can also lead to overfitting, especially with a small dataset.

Task 2) Activation Function Comparison

```
[11]: mlp_relu = Pipeline([("preprocessor", preprocessor),
    ("classifier", MLPClassifier( hidden_layer_sizes=(32,),
        activation="relu",
        max_iter=300,
        random_state=42
    ))
])
mlp_relu.fit(X_train, y_train)
```

/opt/anaconda3/lib/python3.13/site-

packages/sklearn/neural_network/_multilayer_perceptron.py:691:

ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization hasn't converged yet.

warnings.warn(

```
[11]: Pipeline(steps=[('preprocessor',
    ColumnTransformer(transformers=[('num', StandardScaler(),
        ['age', 'anaemia',
        'creatinine_phosphokinase',
        'diabetes',
        'ejection_fraction',
        'high_blood_pressure',
        'platelets',
        'serum_creatinine',
        'serum_sodium', 'sex',
        'smoking', 'time'])])),
    ('classifier',
    MLPClassifier(hidden_layer_sizes=(32,), max_iter=300,
        random_state=42))])
```

```
[12]: relu_pred = mlp_relu.predict(X_test)
relu_prob = mlp_relu.predict_proba(X_test)[: ,1]

print("ReLU Accuracy:", accuracy_score(y_test, relu_pred))
print("ReLU ROC-AUC:", roc_auc_score(y_test, relu_prob))
print("ReLU LogLoss:", log_loss(y_test, relu_prob))
```

ReLU Accuracy: 0.7777777777777778

ReLU ROC-AUC: 0.8133640552995391

ReLU LogLoss: 0.5206469351645078

```
[13]: mlp_tanh = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", MLPClassifier(hidden_layer_sizes=(32,),
        activation="tanh",
        max_iter=300,
        random_state=42
    )))
mlp_tanh.fit(X_train, y_train)
```

/opt/anaconda3/lib/python3.13/site-

packages/sklearn/neural_network/_multilayer_perceptron.py:691:

ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization hasn't converged yet.

warnings.warn(

```
[13]: Pipeline(steps=[('preprocessor',
    ColumnTransformer(transformers=[('num', StandardScaler(),
    ['age', 'anaemia',
    'creatinine_phosphokinase',
    'diabetes',
    'ejection_fraction',
    'high_blood_pressure',
    'platelets',
    'serum_creatinine',
    'serum_sodium', 'sex',
    'smoking', 'time'])])),
    ('classifier',
    MLPClassifier(activation='tanh', hidden_layer_sizes=(32,),
        max_iter=300, random_state=42))])
```

```
[14]: tanh_pred = mlp_tanh.predict(X_test)
tanh_prob = mlp_tanh.predict_proba(X_test)[: ,1]

print("tanh Accuracy:", accuracy_score(y_test, tanh_pred))
print("tanh ROC-AUC:", roc_auc_score(y_test, tanh_prob))
print("tanh LogLoss:", log_loss(y_test, tanh_prob))
```

tanh Accuracy: 0.8222222222222222

```
tanh ROC-AUC: 0.815668202764977
tanh LogLoss: 0.5101720309882067
```

```
[15]: results = pd.DataFrame({
    "Model": ["MLP ReLU", "MLP tanh"],
    "Accuracy": [
        accuracy_score(y_test, relu_pred),
        accuracy_score(y_test, tanh_pred)],
    "ROC-AUC": [
        roc_auc_score(y_test, relu_prob),
        roc_auc_score(y_test, tanh_prob)],
    "LogLoss": [
        log_loss(y_test, relu_prob),
        log_loss(y_test, tanh_prob)]
})
results
```

```
[15]:      Model  Accuracy  ROC-AUC  LogLoss
0  MLP ReLU  0.777778  0.813364  0.520647
1  MLP tanh  0.822222  0.815668  0.510172
```

Both models use the same neural network architecture with different activation functions. The ReLU activation function typically converges faster and performs well because it avoids the vanishing gradient problem. While, the tanh activation function outputs values between -1 and 1 , which can sometimes slow training due to gradient saturation.

In this experiment, the results show that the ReLU model achieved slightly better performance and more stable convergence compared to the tanh model.

Task 3) Capacity and Overfitting

```
[16]: small_mlp = Pipeline([("preprocessor", preprocessor),
    ("classifier", MLPClassifier(hidden_layer_sizes=(8,),
        activation="relu",
        max_iter=300,
        random_state=42))
])
small_mlp.fit(X_train, y_train)
```

```
/opt/anaconda3/lib/python3.13/site-
packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and
the optimization hasn't converged yet.
    warnings.warn(
```

```
[16]: Pipeline(steps=[('preprocessor',
    ColumnTransformer(transformers=[('num', StandardScaler(),
        ['age', 'anaemia',
        'creatinine_phosphokinase',
        'diabetes',
```

```

        'ejection_fraction',
        'high_blood_pressure',
        'platelets',
        'serum_creatinine',
        'serum_sodium', 'sex',
        'smoking', 'time']]])),
    ('classifier',
     MLPClassifier(hidden_layer_sizes=(8,), max_iter=300,
                    random_state=42)))

```

```

[21]: large_mlp = Pipeline([("preprocessor", preprocessor),
    ("classifier", MLPClassifier(hidden_layer_sizes=(128,),
        activation="relu",
        max_iter=300,
        random_state=42))
    ])
large_mlp.fit(X_train, y_train)

```

/opt/anaconda3/lib/python3.13/site-packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and the optimization hasn't converged yet.
warnings.warn(

```

[21]: Pipeline(steps=[('preprocessor',
    ColumnTransformer(transformers=[('num', StandardScaler(),
        ['age', 'anaemia',
        'creatinine_phosphokinase',
        'diabetes',
        'ejection_fraction',
        'high_blood_pressure',
        'platelets',
        'serum_creatinine',
        'serum_sodium', 'sex',
        'smoking', 'time']]])),
    ('classifier',
     MLPClassifier(hidden_layer_sizes=(128,), max_iter=300,
                    random_state=42))])

```

```

[22]: large_mlp_l2 = Pipeline([("preprocessor", preprocessor),
    ("classifier", MLPClassifier(hidden_layer_sizes=(128,),
        activation="relu",
        alpha=0.01,
        max_iter=300,
        random_state=42))
    ])
large_mlp_l2.fit(X_train, y_train)

```

```

/opt/anaconda3/lib/python3.13/site-
packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (300) reached and
the optimization hasn't converged yet.
    warnings.warn(

```

```

[22]: Pipeline(steps=[('preprocessor',
                        ColumnTransformer(transformers=[('num', StandardScaler(),
                                                         ['age', 'anaemia',
                                                         'creatinine_phosphokinase',
                                                         'diabetes',
                                                         'ejection_fraction',
                                                         'high_blood_pressure',
                                                         'platelets',
                                                         'serum_creatinine',
                                                         'serum_sodium', 'sex',
                                                         'smoking', 'time'])])),
                    ('classifier',
                     MLPClassifier(alpha=0.01, hidden_layer_sizes=(128,),
                                     max_iter=300, random_state=42)))]

```

```

[23]: def get_metrics(model, X_train, y_train, X_val, y_val, X_test, y_test):
    train_pred = model.predict(X_train)
    val_pred = model.predict(X_val)
    test_pred = model.predict(X_test)

    train_prob = model.predict_proba(X_train)[:, 1]
    val_prob = model.predict_proba(X_val)[:, 1]
    test_prob = model.predict_proba(X_test)[:, 1]

    return {
        "Train Accuracy": accuracy_score(y_train, train_pred),
        "Validation Accuracy": accuracy_score(y_val, val_pred),
        "Test Accuracy": accuracy_score(y_test, test_pred),
        "Train ROC-AUC": roc_auc_score(y_train, train_prob),
        "Validation ROC-AUC": roc_auc_score(y_val, val_prob),
        "Test ROC-AUC": roc_auc_score(y_test, test_prob),
        "Train LogLoss": log_loss(y_train, train_prob),
        "Validation LogLoss": log_loss(y_val, val_prob),
        "Test LogLoss": log_loss(y_test, test_prob)
    }

results_task3 = pd.DataFrame([
    {"Model": "Small MLP (8 units)", **get_metrics(small_mlp, X_train, y_train,
↪X_val, y_val, X_test, y_test)},
    {"Model": "Large MLP (128 units)", **get_metrics(large_mlp, X_train,
↪y_train, X_val, y_val, X_test, y_test)},

```



```

    {"Model": "Large MLP + L2", **get_metrics(large_mlp_l2, X_train, y_train,
↪X_val, y_val, X_test, y_test)}
])

results_task3

```

```

[23]:
      Model  Train Accuracy  Validation Accuracy  Test Accuracy \
0   Small MLP (8 units)      0.837321      0.733333      0.711111
1  Large MLP (128 units)      0.928230      0.777778      0.755556
2    Large MLP + L2      0.928230      0.777778      0.755556

      Train ROC-AUC  Validation ROC-AUC  Test ROC-AUC  Train LogLoss \
0      0.919908      0.826667      0.732719      0.377330
1      0.978453      0.893333      0.792627      0.208306
2      0.977717      0.893333      0.792627      0.211602

      Validation LogLoss  Test LogLoss
0      0.474293      0.554268
1      0.396126      0.605496
2      0.391623      0.597603

```

Capacity and Overfitting Analysis

Based on the calculated data, small network has limited capacity, so it is possible to underfit the data and fail to capture more complex patterns. On the other hand, larger network has much greater capacity, which can improve training performance, but in case of small datasets the risk of overfitting increases .

This can be seen when the large model achieves better training performance than the small model, but the validation performance does not improve by the same amount or becomes less stable. That pattern shows the model is fitting the training data too closely.

We use L2 regularisation to control this that penalises large weights, encourages a simpler and more generalisable solution. If the regularised large model shows slightly lower training performance but better or more stable validation performance, this indicates that regularisation reduced overfitting.

Task 4) Responsible Evaluation

```

[24]: final_models = {
      "Logistic Regression": log_reg_model,
      "Best Activation MLP": mlp_relu,
      "Best Regularised MLP": large_mlp_l2
    }

final_results = []

for name, model in final_models.items():
    test_pred = model.predict(X_test)
    test_prob = model.predict_proba(X_test)[: , 1]

```

```

final_results.append({
    "Model": name,
    "Test Accuracy": accuracy_score(y_test, test_pred),
    "Test ROC-AUC": roc_auc_score(y_test, test_prob),
    "Test LogLoss": log_loss(y_test, test_prob)
})

final_results_df = pd.DataFrame(final_results)
final_results_df

```

```

[24]:

```

	Model	Test Accuracy	Test ROC-AUC	Test LogLoss
0	Logistic Regression	0.822222	0.806452	0.501492
1	Best Activation MLP	0.777778	0.813364	0.520647
2	Best Regularised MLP	0.755556	0.792627	0.597603

Responsible Evaluation

The best models are selected using validation performance. After the architecture and hyperparameters are chosen, the selected models are evaluated once on the test set to obtain an unbiased estimate of generalisation performance.

The training set is used to fit model parameters, which means the model learns patterns from this data. The validation set is used to compare architectures and hyperparameters, such as activation function, hidden layer size, and regularisation strength. This allows model selection without directly using the test set.

The test set should only be used once after all architectural decisions have been finalised. If the test set is used to choose models, it stops being an unbiased estimate of generalisation performance and effectively becomes part of the model selection process.

There is no need to over-explained small numerical differences between models on such a small dataset like this one. A difference of a few percentage points may be caused by sampling variation rather than a truly better model. Therefore, results should be described cautiously and alongside the broader training and validation behaviour.

```
[ ]:
```